

IME 775 – Lecture 15

Training Neural Networks: Activation Functions, Linear Layers, and Forward Propagation

1. From Heaviside to Differentiable Activations

In Chapter 7, we built neural networks using the **Heaviside step function** $\theta(x)$. This was sufficient for manually designing weights (AND, OR, XOR gates), but it has a fatal flaw for automated training:

The Heaviside step function is not differentiable at $x = 0$ and has zero derivative everywhere else.

Why does this matter? Training a neural network requires computing **gradients** of a loss function with respect to weights. Gradients are vectors of partial derivatives. If the activation function is not differentiable, we cannot compute these gradients – and without gradients, we cannot train.

Solution: Replace the Heaviside step with smooth, differentiable functions that approximate the same 0-to-1 transition.

2. The Sigmoid Function

Definition

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The sigmoid produces an S-shaped curve that smoothly transitions from 0 to 1.

Properties	
Property	Value
Range	$(0, 1)$ — always between 0 and 1
$\sigma(0)$	0.5
$\sigma(x) \rightarrow 1$ as $x \rightarrow +\infty$	Mimics step function for large positive inputs
$\sigma(x) \rightarrow 0$ as $x \rightarrow -\infty$	Mimics step function for large negative inputs
Monotonically increasing	Always
Differentiable	Everywhere

Alternative form with positive exponent

$$\sigma(x) = \frac{e^x}{1 + e^x}$$

This is obtained by multiplying numerator and denominator of the original by e^x .

Symmetry property

$$\sigma(-x) = 1 - \sigma(x)$$

Proof:

$$\sigma(-x) = \frac{1}{1 + e^x} = \frac{e^{-x}}{e^{-x} + 1} = 1 - \frac{1}{1 + e^{-x}} = 1 - \sigma(x)$$

The Derivative of Sigmoid — A Beautiful Result

$$\frac{d\sigma}{dx} = \sigma(x) \cdot (1 - \sigma(x))$$

Proof:

$$\frac{d}{dx} \left(\frac{1}{1 + e^{-x}} \right) = \frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1}{1 + e^{-x}} \cdot \frac{e^{-x}}{1 + e^{-x}} = \sigma(x) \cdot (1 - \sigma(x))$$

This is remarkable: **the derivative of the sigmoid can be expressed purely in terms of the sigmoid itself**. This means once we have computed $\sigma(x)$ during the forward pass, we get the derivative for free — no additional computation needed. This property is why sigmoid was historically the default activation function.

Maximum derivative value

The derivative $\sigma(x)(1 - \sigma(x))$ is maximized when $\sigma(x) = 0.5$, i.e., at $x = 0$:

$$\max \frac{d\sigma}{dx} = 0.5 \times 0.5 = 0.25$$

The derivative is always ≤ 0.25 . This will become important when we discuss the **vanishing gradient problem**.

Workout: Compute $\sigma(0)$, $\sigma(2)$, $\sigma(-3)$, and the derivative $\sigma'(2)$.

Solution:

- $\sigma(0) = \frac{1}{1+e^0} = \frac{1}{2} = 0.5$
- $\sigma(2) = \frac{1}{1+e^{-2}} = \frac{1}{1+0.1353} = \frac{1}{1.1353} \approx 0.8808$
- $\sigma(-3) = 1 - \sigma(3) = 1 - \frac{1}{1+e^{-3}} \approx 1 - 0.9526 = 0.0474$
- $\sigma'(2) = \sigma(2)(1 - \sigma(2)) = 0.8808 \times 0.1192 \approx 0.1050$

Parametrized Sigmoid

$$\sigma(x; w, b) = \frac{1}{1 + e^{-(wx+b)}}$$

- w controls the **steepness**: larger $|w| \rightarrow$ steeper transition (closer to Heaviside)
- b controls the **position**: shifts the transition point along the x -axis

For very large w , the parametrized sigmoid becomes virtually indistinguishable from the Heaviside step function while remaining differentiable.

Workout: For the parametrized sigmoid with $w = 5$ and $b = 0$, compute the output at $x = 0$ and $x = 0.5$.

Solution:

- At $x = 0$: $\sigma(5 \cdot 0 + 0) = \sigma(0) = 0.5$
- At $x = 0.5$: $\sigma(5 \cdot 0.5 + 0) = \sigma(2.5) = \frac{1}{1+e^{-2.5}} \approx \frac{1}{1.0821} \approx 0.924$

The steep slope (high w) causes a rapid transition from 0 to 1 near $x = 0$.

3. The Tanh Function

Definition

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Relationship to Sigmoid

$$\tanh(x) = 2\sigma(2x) - 1$$

Tanh is simply the sigmoid **rescaled** to the range $[-1, 1]$ and **centered** at 0.

Properties

Property	Sigmoid $\sigma(x)$	Tanh $\tanh(x)$
Range	$(0, 1)$	$(-1, 1)$
Output at $x = 0$	0.5	0
Centered?	No (centered at 0.5)	Yes (centered at 0)
Max derivative	0.25 (at $x = 0$)	1.0 (at $x = 0$)

The Derivative of Tanh

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x)$$

Like sigmoid, the derivative of tanh is expressed in terms of tanh itself.

Why Tanh Over Sigmoid?

Nuanced point — gradient strength determines training speed:

Near $x = 0$, the tanh derivative is **4 times larger** than the sigmoid derivative (1.0 vs 0.25). Stronger gradients mean larger weight updates, which means **faster convergence** during training. This advantage is most pronounced when data is centered around 0 — which is why we typically **standardize** inputs (zero mean, unit variance) before feeding them into a neural network.

However, both sigmoid and tanh suffer from the **vanishing gradient problem**: for inputs far from 0 (the "saturation" regions), the derivative approaches 0, causing gradients to nearly vanish during backpropagation through many layers. This is why modern networks often use **ReLU** (Rectified Linear Unit, covered in Ch. 9), which does not saturate for positive inputs.

Workout: Compute $\tanh(0)$, $\tanh(1)$, and $\tanh'(1)$.

Solution:

- $\tanh(0) = \frac{e^0 - e^0}{e^0 + e^0} = \frac{0}{2} = 0$
 - $\tanh(1) = \frac{e - e^{-1}}{e + e^{-1}} = \frac{2.718 - 0.368}{2.718 + 0.368} = \frac{2.350}{3.086} \approx 0.762$
 - $\tanh'(1) = 1 - \tanh^2(1) = 1 - 0.762^2 = 1 - 0.581 \approx 0.419$
-

4. Why Layering?

In Chapter 7, we saw that multiple perceptrons allow us to model problems a single perceptron cannot solve (XOR). Cybenko's theorem guarantees that a **single hidden layer** can approximate any continuous function — so why use more than one layer?

The Power of Depth

Each layer introduces its own **nonlinear activation function**. More nonlinearities stacked together can model more complicated functions with **fewer total neurons** than a single wide layer.

Architecture	Nonlinearities	Practical expressiveness
Single perceptron	1	Linear decision boundary only
1 hidden layer, n neurons	n	Any function (Cybenko), but may need impractically many neurons
L hidden layers, m neurons each	$L \times m$	Same expressiveness with far fewer parameters

Deeper explanation:

Consider approximating a function that oscillates n times. A single hidden layer requires $O(n)$ neurons (one tower per oscillation). A deep network can compose primitive features: layer 1 detects basic patterns, layer 2 combines them, layer 3 combines the combinations, and so on. This compositional structure achieves the same approximation with $O(\log n)$ neurons per layer — exponentially fewer total parameters.

5. Linear (Fully Connected) Layers

Definition

In a **linear layer** (also called **fully connected layer** or **dense layer**), every neuron in the previous layer is connected to every neuron in the current layer. If layer $l - 1$ has m neurons and layer l has n neurons, there are $m \times n$ connections, each with its own weight.

Splitting a Neuron into Two Operations

In the MLP formalism, each neuron performs two operations:

1. **Weighted sum** (affine transformation): $z_j^{(l)} = \sum_{k=0}^m w_{jk}^{(l)} a_k^{(l-1)} + b_j^{(l)}$
2. **Activation** (nonlinearity): $a_j^{(l)} = f(z_j^{(l)})$

We split these into separate "sub-layers" for clarity:

```
a^(l-1) → [Weighted Sum: z = Wa + b] → [Activation: a = f(z)]
→ a^(l)
```

Weight Indexing Convention

The weight $w_{jk}^{(l)}$ connects neuron k in layer $l - 1$ to neuron j in layer l :

- **First subscript (j): destination** neuron
- **Second subscript (k): source** neuron
- **Superscript (l): layer** the weight belongs to

This "destination first" convention is counterintuitive but standard because it makes the matrix form clean.

6. Matrix-Vector Form of Linear Layers

The Compact Equation

For all neurons in layer l simultaneously:

$$\vec{z}^{(l)} = W^{(l)}\vec{a}^{(l-1)} + \vec{b}^{(l)}$$

$$\vec{a}^{(l)} = f\left(\vec{z}^{(l)}\right)$$

where:

- $W^{(l)}$ is an $n \times m$ **weight matrix** (row j contains weights feeding into neuron j)
- $\vec{a}^{(l-1)}$ is the m -dimensional **activation vector** from the previous layer
- $\vec{b}^{(l)}$ is the n -dimensional **bias vector**
- $\vec{z}^{(l)}$ is the n -dimensional **pre-activation** vector
- $f(\cdot)$ is applied **elementwise** to $\vec{z}^{(l)}$

The Weight Matrix

$$W^{(l)} = \begin{pmatrix} w_{00}^{(l)} & w_{01}^{(l)} & \cdots & w_{0m}^{(l)} \\ w_{10}^{(l)} & w_{11}^{(l)} & \cdots & w_{1m}^{(l)} \\ \vdots & & \ddots & \vdots \\ w_{n0}^{(l)} & w_{n1}^{(l)} & \cdots & w_{nm}^{(l)} \end{pmatrix}$$

Row j of $W^{(l)}$ contains all the weights feeding into neuron j of layer l . The matrix-vector product $W^{(l)}\vec{a}^{(l-1)}$ simultaneously computes the dot product of each row with the input, giving all pre-activations at once.

Connection to Chapter 2: This is simply matrix-vector multiplication (Section 2.7) applied to neural networks. The linear algebra tools we built earlier are now doing the heavy lifting.

Workout: A layer has 3 input neurons and 2 output neurons. The weight matrix and bias are:

$$W^{(l)} = \begin{pmatrix} 0.5 & -0.3 & 0.8 \\ 0.2 & 0.7 & -0.4 \end{pmatrix}, \quad \vec{b}^{(l)} = \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix}$$

Compute $\vec{z}^{(l)}$ and $\vec{a}^{(l)}$ (using sigmoid) for input $\vec{a}^{(l-1)} = (1, 0, 1)^T$.

Solution:

$$\vec{z}^{(l)} = \begin{pmatrix} 0.5 & -0.3 & 0.8 \\ 0.2 & 0.7 & -0.4 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix} = \begin{pmatrix} 0.5 + 0 + 0.8 \\ 0.2 + 0 - 0.4 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix} = \begin{pmatrix} 1.4 \\ -0.4 \end{pmatrix}$$

$$\vec{a}^{(l)} = \sigma(\vec{z}^{(l)}) = \begin{pmatrix} \sigma(1.4) \\ \sigma(-0.4) \end{pmatrix} \approx \begin{pmatrix} 0.802 \\ 0.401 \end{pmatrix}$$

7. Forward Propagation

The Algorithm

Given an input $\vec{x}$ and an MLP with $L + 1$ layers (layers 0 through L), forward propagation evaluates the network output by processing one layer at a time:

$$\vec{a}^{(0)} = f\left(W^{(0)}\vec{x} + \vec{b}^{(0)}\right)$$

$$\vec{a}^{(1)} = f\left(W^{(1)}\vec{a}^{(0)} + \vec{b}^{(1)}\right)$$

$$\vdots$$

$$\vec{a}^{(L)} = f\left(W^{(L)}\vec{a}^{(L-1)} + \vec{b}^{(L)}\right) = \vec{y}$$

The final output $\vec{a}^{(L)} = \vec{y}$ is the network's prediction.

The Grand Output Function

The full network computes:

$$\text{MLP}(\vec{x}) = f \left(W^{(L)} \dots f \left(W^{(1)} f \left(W^{(0)} \vec{x} + \vec{b}^{(0)} \right) + \vec{b}^{(1)} \right) \dots + \vec{b}^{(L)} \right)$$

This nested expression is never evaluated directly. Instead, we compute **one layer at a time**, passing the output of each layer as input to the next.

Why Layer-by-Layer is Elegant

At any step, we only need the **previous layer's activation** to compute the **current layer's activation**. This means:

- Memory efficient: only two layers in memory at a time
- Simple to implement: repeat the same two-line computation
- Easy to extend: adding a layer is just one more iteration

Workout: Trace the forward pass through a complete MLP with architecture $2 \rightarrow 2 \rightarrow 1$ (2 inputs, 2 hidden neurons, 1 output) with sigmoid activations.

$$W^{(0)} = \begin{pmatrix} 0.5 & 0.4 \\ 0.3 & -0.2 \end{pmatrix}, \quad \vec{b}^{(0)} = \begin{pmatrix} -0.1 \\ 0.2 \end{pmatrix}$$

$$\vec{w}^{(1)} = (0.6 \quad -0.5), \quad b^{(1)} = 0.1$$

Input: $\vec{x} = (1, 0.5)^T$.

Solution:

Layer 0:

$$\vec{z}^{(0)} = \begin{pmatrix} 0.5 & 0.4 \\ 0.3 & -0.2 \end{pmatrix} \begin{pmatrix} 1 \\ 0.5 \end{pmatrix} + \begin{pmatrix} -0.1 \\ 0.2 \end{pmatrix} = \begin{pmatrix} 0.5 + 0.2 \\ 0.3 - 0.1 \end{pmatrix} + \begin{pmatrix} -0.1 \\ 0.2 \end{pmatrix} = \begin{pmatrix} 0.6 \\ 0.4 \end{pmatrix}$$

$$\vec{a}^{(0)} = \sigma(\vec{z}^{(0)}) = \begin{pmatrix} \sigma(0.6) \\ \sigma(0.4) \end{pmatrix} \approx \begin{pmatrix} 0.646 \\ 0.599 \end{pmatrix}$$

Layer 1:

$$z^{(1)} = \begin{pmatrix} 0.6 & -0.5 \end{pmatrix} \begin{pmatrix} 0.646 \\ 0.599 \end{pmatrix} + 0.1 = 0.388 - 0.300 + 0.1 = 0.188$$

$$y = a^{(1)} = \sigma(0.188) \approx 0.547$$

The network outputs ≈ 0.547 for input $(1, 0.5)$.

Key Takeaways

1. The **sigmoid** $\sigma(x) = \frac{1}{1+e^{-x}}$ is a smooth, differentiable replacement for the Heaviside step function
2. Sigmoid derivative: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ – computed for free from the forward pass output
3. **Tanh** is centered at 0 with range $(-1, 1)$ and $4\times$ stronger gradients near 0 than sigmoid
4. Both sigmoid and tanh **saturate** for large $|x|$, causing vanishing gradients in deep networks
5. **Linear layers** compute $\vec{z} = W\vec{a} + \vec{b}$ followed by elementwise activation $\vec{a} = f(\vec{z})$
6. **Forward propagation** evaluates the network one layer at a time, from input to output
7. The entire forward pass is just repeated matrix-vector multiply $\rightarrow$ activate $\rightarrow$ pass forward

PyTorch Connection

```
import torch
import torch.nn as nn

# Sigmoid
x = torch.tensor([-3.0, -1.0, 0.0, 1.0, 3.0])
sig = torch.sigmoid(x)
print(f"sigmoid: {sig}")          # ≈ [0.047, 0.269, 0.500, 0.731, 0.953]
print(f"derivative: {sig * (1-sig)}") # ≈ [0.045, 0.197, 0.250, 0.197, 0.045]

# Tanh
tanh_out = torch.tanh(x)
print(f"tanh: {tanh_out}")        # ≈ [-0.995, -0.762, 0.000, 0.762, 0.995]
print(f"derivative: {1 - tanh_out**2}") # ≈ [0.010, 0.420, 1.000, 0.420, 0.010]

# Forward propagation through a 2-layer MLP
model = nn.Sequential(
    nn.Linear(3, 4),    # Layer 0: 3 inputs → 4 hidden
    nn.Sigmoid(),
    nn.Linear(4, 2),    # Layer 1: 4 hidden → 2 hidden
    nn.Sigmoid(),
    nn.Linear(2, 1),    # Layer 2: 2 hidden → 1 output
)

x_input = torch.tensor([[1.0, 0.5, -0.3]])
y_pred = model(x_input) # Forward propagation happens automatically
print(f"Output: {y_pred}")
```